Northeastern University

Class Assignment 4: EECE 7205

Professor: Dr. Fatema Nafa

Rohini Bidnurmath
NUID: 003916201

October 2026

Contents

1 Problem 1: Building the Tree (Insert) 2

2 Problem 2: Searching the Tree 7

3 Problem 3: Deleting from the Tree 11

1 Problem 1: Building the Tree (Insert)

1.3 Case 3: Insert a larger key (insert 70)

(a) Rule(s) and Explanation:

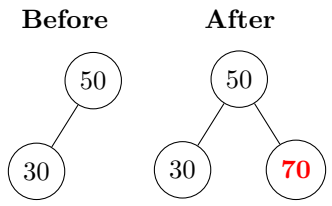
- **Rules applied:** I3 then I1.
- **Explanation:** Compare 70 with the root (50). Since $70 > 50$, the key belongs in the right subtree (I3). The right child of 50 is NULL, so an empty spot is found (I1), and 70 is attached as the right child of 50. Exactly 1 comparison was needed.

(b) Pseudocode:

Algorithm 1 Insert a larger key (Case 3)

```
1: cur ← root
2: if key > cur.key then                          ▷ I3: go right
3:     if cur.right = NULL then                   ▷ I1: empty spot found
4:         cur.right ← newNode
5:         return
6:     end if
7:     cur ← cur.right
8: end if
```

(c) Tree Before and After:



1.4 Case 4: Insert deep in the tree (insert $20, 40, 60, 80$, then 35)

(a) Rule(s) and Explanation: Inserting $20, 40, 60, 80$ fills level 2 (20 and 40 under 30; 60 and 80 under 70). To insert 35:

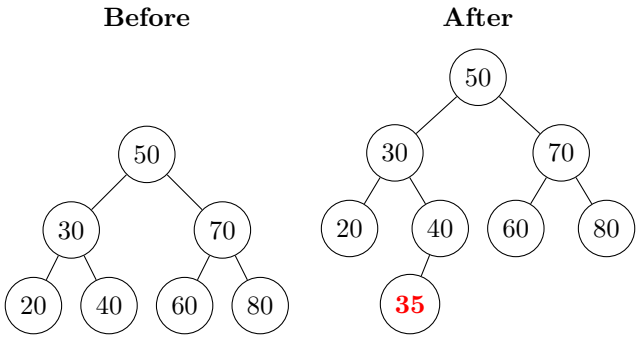
- Compare 35 with 50: $35 < 50 \rightarrow$ go left (I2).
- Compare 35 with 30: $35 > 30 \rightarrow$ go right (I3).
- Compare 35 with 40: $35 < 40 \rightarrow$ go left (I2).
- The left spot of 40 is NULL $\rightarrow$ attach 35 as the left child of 40 (I1).
- **Nodes compared in order:** $50, 30, 40$.
- **Why left at one node and right at another:** at every node the decision depends only on that node's key. 35 is smaller than 50 and 40 but larger than 30, so the path turns: left at 50, right at 30, left at 40. This places 35 in the only position where it is in 50's left subtree, 30's right subtree and 40's left subtree at the same time.

(b) Pseudocode (Loop Walking Down):

Algorithm 2 Walk down the tree and attach the new node (Case 4)

```
parent ← NULL
cur ← root
while cur ≠ NULL do
    parent ← cur
    if key < cur.key then                          ▷ I2: go left
        cur ← cur.left
    else if key > cur.key then                     ▷ I3: go right
        cur ← cur.right
    else                                           ▷ I4: duplicate, stop
        return
    end if
end while
if key < parent.key then                           ▷ I1: empty spot on the left
    parent.left ← newNode
else                                               ▷ I1: empty spot on the right
    parent.right ← newNode
end if
```

(c) Tree Before and After Inserting 35:



1.5 Case 5: Insert a key already in the tree (insert 40 again)

(a) Rule(s) and Explanation: First, complete the tree by inserting 45 (right child of 40), 65 (right child of 60) and 75 (left child of 80). Attempting to insert 40 again:

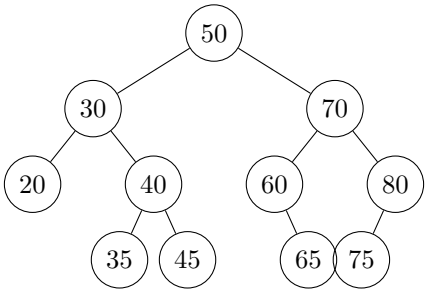
- Compare with 50: $40 < 50 \rightarrow$ go left (I2).
- Compare with 30: $40 > 30 \rightarrow$ go right (I3).
- Compare with 40: $40 = 40 \rightarrow$ duplicate found (I4). Nothing is inserted and the tree is unchanged.
- **Why duplicates are not stored:** the BST property is strict ($k_{\text{left}} < k_{\text{node}} < k_{\text{right}}$), so a second 40 would have no valid position: it is neither smaller nor larger than the existing 40. Allowing it would break the invariant that search, insert and delete all rely on, and a search for 40 could return either copy.

(b) Pseudocode:

Algorithm 3 Detect a duplicate key (Case 5)

```
1: cur ← root
2: while cur ≠ NULL do
3:     if key = cur.key then                          ▷ I4: duplicate key
4:         return                                     ▷ do nothing
5:     else if key < cur.key then                     ▷ I2
6:         cur ← cur.left
7:     else                                           ▷ I3
8:         cur ← cur.right
9:     end if
10: end while
```

(c) Final Tree of `arr` (All 11 Keys):



1.6 Case 6: Insert keys arriving in sorted order $(20, 30, 40, 50, 60)$

(a) Rule(s) and Explanation:

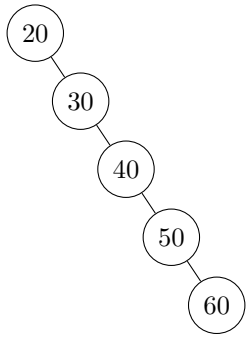
- **Rules applied:** 20 goes into the empty tree (I1). Every later key uses I3 at every node it meets, then I1.
- **Explanation:** because the keys arrive in ascending order, each new key is larger than every key already in the tree, so it always goes right (I3) until it reaches NULL (I1). No key ever goes left.
- **Height comparison:** the height is $h = 4$ (edges) for only 5 keys, the maximum possible $n - 1$. The Case 5 tree holds 11 keys with height only 3. Sorted input makes the BST degenerate into a linked list, so search costs $O(n)$ instead of $O(\log n)$.

(b) Pseudocode BUILD-FROM-ARRAY(data, n):

Algorithm 4 Build a BST from an array

```
1: function BUILD-FROM-ARRAY(data, n)
2:     root ← NULL
3:     for i ← 0 to n − 1 do
4:         INSERT(data[i])                            ▷ Algorithm 5
5:     end for
6:     return root
7: end function
```

(c) Tree Diagram:



(g) Table for Inserting All of `arr`

Step	Insert	Nodes compared (in order)	Rule(s)	Parent	Left / Right
1	50	none	I1	none	root
2	30	50	I2, I1	50	left
3	70	50	I3, I1	50	right
4	20	50, 30	I2, I2, I1	30	left
5	40	50, 30	I2, I3, I1	30	right
6	60	50, 70	I3, I2, I1	70	left
7	80	50, 70	I3, I3, I1	70	right
8	35	50, 30, 40	I2, I3, I2, I1	40	left
9	45	50, 30, 40	I2, I3, I3, I1	40	right
10	65	50, 70, 60	I3, I2, I3, I1	60	right
11	75	50, 70, 80	I3, I3, I2, I1	80	left

(h) Combined INSERT(key) Pseudocode & Metrics

Algorithm 5 Complete INSERT(key), combining Cases 1–5

```
1:  function INSERT(key)
2:      if root = NULL then                                  ▷ I1: empty tree
3:          root ← new node with key
4:          return
5:      end if
6:      cur ← root
7:      loop
8:          if key = cur.key then                            ▷ I4: duplicate, do nothing
9:              return
10:         else if key < cur.key then                       ▷ I2: go left
11:             if cur.left = NULL then                      ▷ I1: empty spot
12:                 cur.left ← new node with key
13:                 return
14:             end if
15:             cur ← cur.left
16:         else                                             ▷ I3: go right
17:             if cur.right = NULL then                     ▷ I1: empty spot
18:                 cur.right ← new node with key
19:                 return
20:             end if
21:             cur ← cur.right
22:         end if
23:     end loop
24: end function
```

- **Height of final tree:** $h = 3$ (edges on the longest root-to-leaf path, e.g. $50 \to 30 \to 40 \to 35$).
- **Comparisons for the deepest insert:** 3 key comparisons (e.g. 35 is compared with $50, 30, 40$, as in the table), which equals the height of the final tree.

2 Problem 2: Searching the Tree

2.3 Case 3: Target in right subtree, path turns (search 65)

(a) Rule at Each Node and Explanation:

- Node 50: $65 > 50 \rightarrow$ go right (S4).
- Node 70: $65 < 70 \rightarrow$ go left (S3).
- Node 60: $65 > 60 \rightarrow$ go right (S4).
- Node 65: $65 = 65 \rightarrow$ target found (S2).
- **Total comparisons:** 4. The path turns twice (right, left, right); the whole left subtree of 50 is never visited.

(b) Pseudocode:

Algorithm 6 Search loop used in Case 3 (S2, S3, S4)

```
1:  cur ← root
2:  while cur ≠ NULL do
3:      if target = cur.key then                    ▷ S2: found
4:          return FOUND
5:      else if target < cur.key then               ▷ S3: go left
6:          cur ← cur.left
7:      else                                        ▷ S4: go right
8:          cur ← cur.right
9:      end if
10: end while
```

(c) Search Path:

$$50 \xrightarrow{\text{S4}} 70 \xrightarrow{\text{S3}} 60 \xrightarrow{\text{S4}} 65 \quad \text{(found after 4 comparisons)}$$

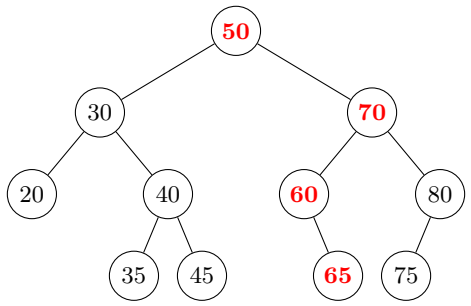


Figure 1: Search for 65: nodes on the path are in red.

2.4 Case 4: Target not in the tree (search 33)

(a) Rule at Each Node and Explanation:

- Node 50: $33 < 50 \rightarrow$ go left (S3).
- Node 30: $33 > 30 \rightarrow$ go right (S4).

- Node 40: $33 < 40 \rightarrow$ go left (S3).
- Node 35: $33 < 35 \rightarrow$ go left (S3).
- 35.left = NULL $\rightarrow$ the search stops there: not found (S1). 4 comparisons.
- **Parent if inserted:** 35 would be the parent of 33, and 33 would be its *left* child, exactly in the NULL spot where the search stopped.

(b) Pseudocode:

Algorithm 7 Unsuccessful search (Case 4, ends with S1)

```
1:  cur ← root
2:  while cur ≠ NULL do
3:      if target = cur.key then                          ▷ S2
4:          return FOUND
5:      else if target < cur.key then                     ▷ S3
6:          cur ← cur.left
7:      else                                              ▷ S4
8:          cur ← cur.right
9:      end if
10: end while
11: return NOT FOUND                                      ▷ S1: reached NULL
```

(c) Search Path:

$$50 \xrightarrow{\text{S3}} 30 \xrightarrow{\text{S4}} 40 \xrightarrow{\text{S3}} 35 \xrightarrow{\text{S3}} \text{NULL} \quad \text{(not found after 4 comparisons)}$$

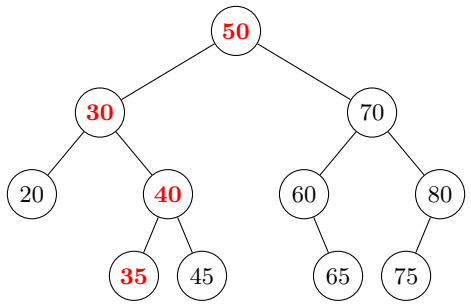


Figure 2: Search for 33: the path (red) ends at the empty left spot of 35.

2.5 Case 5: Target larger than every key (search 90)

(a) Rule at Each Node and Explanation:

- Node 50: $90 > 50 \rightarrow$ go right (S4).
- Node 70: $90 > 70 \rightarrow$ go right (S4).
- Node 80: $90 > 80 \rightarrow$ go right (S4).
- 80.right = NULL $\rightarrow$ not found (S1).
- **Total comparisons:** 3. A target larger than every key always follows the rightmost path and stops just past the maximum.

(b) Pseudocode:

Algorithm 8 Search for a target larger than every key (Case 5)

```
1:  cur ← root
2:  while cur ≠ NULL do
3:      if target = cur.key then                                  ▷ S2
4:          return FOUND
5:      else if target < cur.key then                             ▷ S3
6:          cur ← cur.left
7:      else                                  ▷ S4: taken at every node here
8:          cur ← cur.right
9:      end if
10: end while
11: return NOT FOUND                                              ▷ S1
```

(c) Search Path:

$$50 \xrightarrow{\text{S4}} 70 \xrightarrow{\text{S4}} 80 \xrightarrow{\text{S4}} \text{NULL} \quad \text{(not found after 3 comparisons)}$$

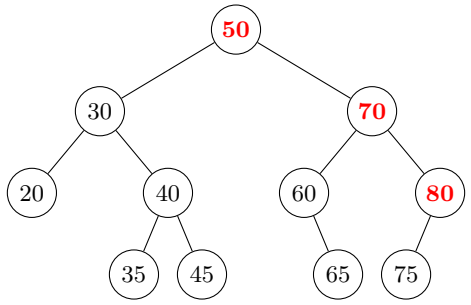


Figure 3: Search for 90: the path (red) follows right pointers and ends at the empty right spot of 80.

2.6 Case 6: Find the minimum and maximum key

(a) Rule and Explanation:

- **Minimum:** start at the root and keep following left pointers; stop at the node whose left child is NULL. That node is 20. Every smaller key would have to be in its left subtree, which is empty.
- **Maximum:** start at the root and keep following right pointers; stop at the node whose right child is NULL. That node is 80 (it has a left child, 75, but no right child).
- No key comparisons are needed: only the pointers are checked.

(b) Pseudocode FIND-MIN and FIND-MAX:

Algorithm 9 FIND-MIN and FIND-MAX (return the node)

```
1: function FIND-MIN(node)
2:     cur ← node
3:     while cur.left ≠ NULL do                    ▷ keep going left
4:         cur ← cur.left
5:     end while
6:     return cur
7: end function

8: function FIND-MAX(node)
9:     cur ← node
10:    while cur.right ≠ NULL do                   ▷ keep going right
11:        cur ← cur.right
12:    end while
13:    return cur
14: end function
```

(c) Search Paths:

- **Min path:** $50 \to 30 \to 20$ (minimum = 20)
- **Max path:** $50 \to 70 \to 80$ (maximum = 80)

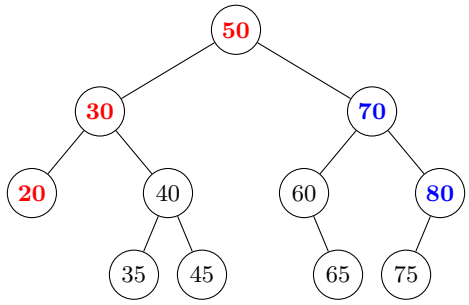


Figure 4: Min path in red ($50 \to 30 \to 20$), max path in blue ($50 \to 70 \to 80$).

2.7 Summary Table for Searching Problem 2

Case	Target	Nodes compared (in order)	Rules	Result	Comparisons
1	50	50	S2	found	1
2	45	50, 30, 40, 45	S3, S4, S4, S2	found	4
3	65	50, 70, 60, 65	S4, S3, S4, S2	found	4
4	33	50, 30, 40, 35	S3, S4, S3, S3, S1	not found	4
5	90	50, 70, 80	S4, S4, S4, S1	not found	3
6	min / max	min: 50, 30, 20; max: 50, 70, 80	left / right	20 / 80	0 (3 nodes visited each)

(g) Maximum Comparisons & Height Relationship: The largest number of comparisons any search can need in this tree is 4 (for example, searching 35, 45, 65, 75, or a missing key such as 33). For a BST of height h (in edges), a search compares with at most one node per level, so the worst case is $h+1 = 3+1 = 4$ comparisons.

3 Problem 3: Deleting from the Tree

3.3 Case 3: Delete a node with only a right child (delete 60)

(a) Rule(s) and Explanation:

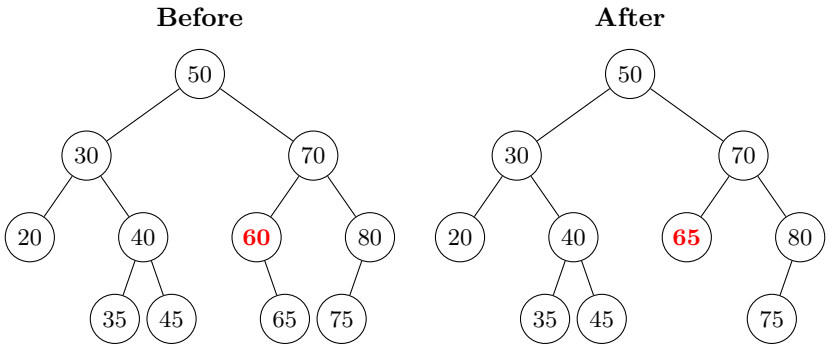
- **Rule applied:** D2 (one child).
- **Explanation:** search $50 \to 70 \to 60$. Node 60 has no left child and one right child (65). Replace 60 by its child: 70.left $\leftarrow$ 65. The BST property still holds because 65 was already in 70's left subtree.

(b) Pseudocode:

Algorithm 10 Delete a node with only a right child (D2)

```
1: if node.left = NULL then                          ▷ D2: only a right child
2:     temp ← node.right
3:     FREE(node)
4:     return temp                                   ▷ parent now points to the child
5: end if
```

(c) Tree Before and After:



3.4 Case 4: Delete a node with only a left child (delete 80)

(a) Rule(s) and Explanation:

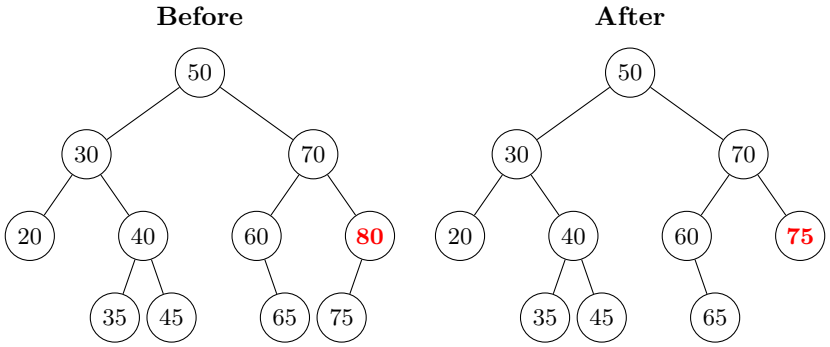
- **Rule applied:** D2 (one child).
- **Explanation:** search $50 \to 70 \to 80$. Node 80 has only a left child (75). Replace 80 by its child: 70.right $\leftarrow$ 75.

(b) Pseudocode:

Algorithm 11 Delete a node with only a left child (D2)

```
1: if node.right = NULL then                         ▷ D2: only a left child
2:     temp ← node.left
3:     FREE(node)
4:     return temp
5: end if
```

(c) Tree Before and After:



3.5 Case 5: Delete node with two children, successor is right child (delete 40)

(a) Rule(s) and Explanation:

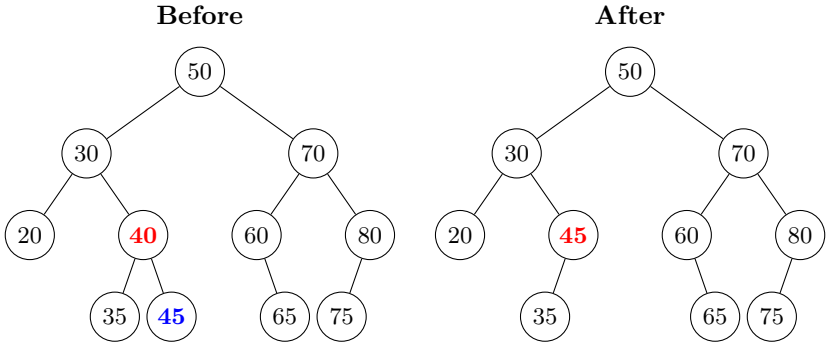
- **Rules applied:** D3 (two children), then D1 (leaf) to delete the successor.
- **Finding the successor:** node 40 has two children (35 and 45). Its inorder successor is found by calling FIND-MIN on the right subtree of 40: go right once to 45, then left as far as possible. 45 has no left child, so the walk stops immediately and the successor is 45.
- **Why it is 40's own right child:** the successor is the leftmost node of the right subtree. Since the right child 45 has no left subtree, nothing in 40's right subtree is smaller than 45, so the right child itself is the minimum.
- Copy 45 into node 40, then delete key 45 from the right subtree. That node is a leaf, so D1 applies.

(b) Pseudocode:

Algorithm 12 Delete a node with two children (D3, successor deleted by D1)

```
succ ← FIND-MIN(node.right)          ▷ D3: inorder successor (45)
node.key ← succ.key                  ▷ copy 45 into the node
node.right ← DELETE(node.right, succ.key)   ▷ 45 is a leaf: D1
```

(c) Tree Before and After:



Red: the node being deleted / its new key. Blue: the inorder successor.

3.6 Case 6: Delete root, successor has a child (delete 50)

(a) Rule(s) and Explanation:

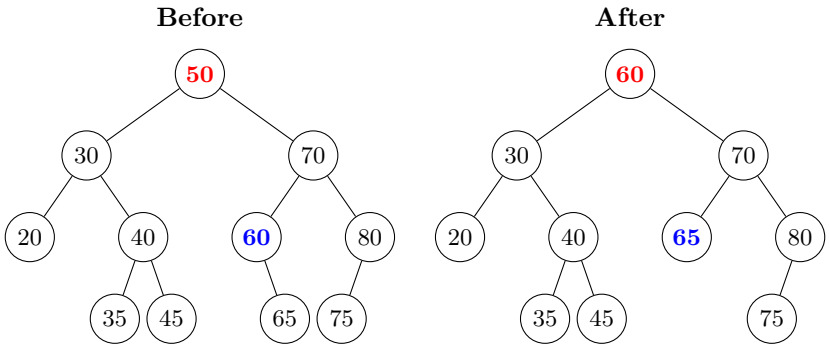
- **Rules applied:** D3 (two children), then D2 (one child) to delete the successor.
- **Finding the successor:** go right once to 70, then left as far as possible: 60, whose left child is NULL. The successor is 60.
- Copy 60 into the root. Then delete key 60 from the right subtree: node 60 has one child (65), so D2 applies and 65 takes 60's place as 70.left.

(b) Pseudocode:

Algorithm 13 Delete the root (D3, successor deleted by D2)

```
succ ← FIND-MIN(node.right)          ▷ succ = node 60
node.key ← succ.key                  ▷ root key becomes 60
node.right ← DELETE(node.right, succ.key)   ▷ 60 has one child: D2
```

(c) Tree Before and After:



Red: the root and its new key. Blue: the successor 60 and the child 65 that replaces it.

3.7 Summary Table for Deletion Problem 3

Case	Delete	Children of node	Successor	Rules applied	Level order after deletion
1	33	not found	–	S1	50 30 70 20 40 60 80 35 45 65 75
2	20	none	–	D1	50 30 70 40 60 80 35 45 65 75
3	60	right only (65)	–	D2	50 30 70 20 40 65 80 35 45 75
4	80	left only (75)	–	D2	50 30 70 20 40 60 75 35 45 65
5	40	two (35, 45)	45	D3, D1	50 30 70 20 45 60 80 35 65 75
6	50	two (30, 70)	60	D3, D2	60 30 70 20 40 65 80 35 45 75

3.8 (g) Complete Combined DELETE(node, key) Pseudocode

Algorithm 14 Complete DELETE(node, key), combining Cases 1–6

```
1:  function DELETE(node, key)                          ▷ returns the new root of this subtree
2:      if node = NULL then                             ▷ Case 1, S1: key not in the tree
3:          return NULL
4:      end if
5:      if key < node.key then
6:          node.left ← DELETE(node.left, key)
7:      else if key > node.key then
8:          node.right ← DELETE(node.right, key)
9:      else                                            ▷ key = node.key: found it
10:         if node.left = NULL and node.right = NULL then   ▷ Case 2, D1: leaf
11:             FREE(node)
12:             return NULL
13:         else if node.left = NULL then               ▷ Case 3, D2: only right child
14:             temp ← node.right
15:             FREE(node)
16:             return temp
17:         else if node.right = NULL then              ▷ Case 4, D2: only left child
18:             temp ← node.left
19:             FREE(node)
20:             return temp
21:         else                                        ▷ Cases 5–6, D3: two children
22:             succ ← FIND-MIN(node.right)
23:             node.key ← succ.key
24:             node.right ← DELETE(node.right, succ.key)
25:         end if
26:     end if
27:     return node
28: end function
```

3.9 (h) Why Copy the Inorder Successor Instead of the Deepest Node?

In a general binary tree there is no ordering between keys, so any node (conventionally the deepest, rightmost one) can fill the hole while keeping the tree complete. A **binary search tree** has a stronger requirement:

1. Every node must satisfy $k_{\text{left}} < k_{\text{node}} < k_{\text{right}}$ for *all* keys in its subtrees, not just its children.

2. The **inorder successor** is the smallest key in the right subtree. Putting it in the deleted node's place keeps it larger than every key in the left subtree (it came from the right side) and smaller than every key left in the right subtree (it was the minimum there).

3. The deepest node can have any key. For example, when deleting the root 50, moving the deepest, rightmost node 75 into the root would leave 60, 65 and 70 in its right subtree even though they are smaller than 75. The BST property breaks, and a later search for 65 would go left at the root and fail.

4. The successor is also easy to delete afterwards: it has no left child, so removing it is always a D1 or D2 case, never another D3.